

# Reviewer Pack — Minimal Automorphic Rank of Affine Line Arrangements and Com...

Entry 5848c55fb4b2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 23:37:47 UTC

## Minimal Automorphic Rank of Affine Line Arrangements and Communication Complexity Growth

Entry ID: 5848c55fb4b2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 23:37:47 UTC

### 1. Conjecture

**Field A** (mathematical branch): Automorphic L-Function Theory **Field B** (complexity object): Communication Complexity

#### Statement:

The minimal automorphic rank of an affine line arrangement in a projective space is linearly correlated with the exponential growth rate of its communication complexity, such that for any affine line arrangement  $A$ , the exponential communication complexity  $c(A) = \Theta(\min_r \rho(A, r))$ , where  $r$  ranges over the possible ranks and  $\rho(A, r)$  is the communication complexity for rank  $r$ .

#### Rationale (proposer's reasoning):

Automorphic L-function theory has rich algebraic structures that could potentially encode combinatorial properties of computational tasks like communication complexity. The minimal automorphic rank captures a fundamental invariant in this theory, which might reveal hidden patterns in the complexity landscape of communication tasks.

**Taxonomy category:** automorphic\_rank\_communication\_complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** b1fcade05d21e4f7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal automorphic rank and the exponential communication complexity is statistically significant ( $p$ -value  $< 0.05$ ) across all arrangements, and falsified if not.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** ADJACENT\_OK against 5 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - title:('Automorphic L-Function Theory' AND 'Communication Complexity') - arxiv:math.AG AND title:communication complexity - subject:automorphic rank AND subject:communication complexity growth rate

**Top relevant hits considered:** - [http://arxiv.org/abs/math/0211278v7] Patchworking singular algebraic curves, non-Archimedean amoebas and enumerative geometry - [http://arxiv.org/abs/0710.1443v2] Variations on themes of Kostant - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
        for i in range(n):
            max_row = max(range(i, n), key=lambda r: abs(augmented_matrix[r][i]))
            augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
            pivot = augmented_matrix[i][i]
            if pivot == 0:
                continue
            for j in range(n + 1):
                augmented_matrix[i][j] /= pivot
            for k in range(n):
```

```

        if k != i and abs(augmented_matrix[k][i]) > 0:
            factor = augmented_matrix[k][i]
            for j in range(n + 1):
                augmented_matrix[k][j] -= factor * augmented_matrix[i][j]
    return [row[-1] for row in augmented_matrix]

def communication_complexity(A, rank):
    n = len(A)
    if rank == 0:
        return 0
    B = [[A[i][j] for j in range(rank)] for i in range(n)]
    c = gaussian_elimination(B, [1] * rank)
    return sum(abs(x) for x in c)

def minimal_automorphic_rank(A):
    n = len(A)
    ranks = set()
    for r in range(1, n + 1):
        B = [[A[i][j] for j in range(r)] for i in range(n)]
        if gaussian_elimination(B, [1] * r) == [0] * r:
            ranks.add(r)
    return min(ranks)

def correlation_coefficient(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = (sum((x[i] - mean_x) ** 2 for i in range(n)) / n) ** 0.5
    std_y = (sum((y[i] - mean_y) ** 2 for i in range(n)) / n) ** 0.5
    return cov_xy / (std_x * std_y)

def generate_random_arrangement(n):
    A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    return A

n_values = [5, 10, 15, 20, 30, 40]
ranks = []
complexities = []

for n in n_values:
    for _ in range(5):
        A = generate_random_arrangement(n)
        rank = minimal_automorphic_rank(A)
        complexity = communication_complexity(A, rank)
        ranks.append(rank)
        complexities.append(complexity)

if not ranks or not complexities:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

```

```

    }

    correlation = correlation_coefficient(ranks, complexities)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation,
        "instances_tested": len(ranks),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation) >= 0.95,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is
    support_fraction = sum(r["conjecture_holds"] for r in results if r["conjecture_holds"]) / len

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=not_enough_data")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9de491fc.py", line 118, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9de491fc.py", line 85, in run_trial
    rank = minimal_automorphic_rank(A)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9de491fc.py", line 61, in minimal_autom
    if gaussian_elimination(B, [1] * r) == [0] * r:
       ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9de491fc.py", line 32, in gaussian_elim
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]

```

~^^^

IndexError: list index out of range

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means it did not complete the computation to determine the correlation coefficient and p-value. | next: Re-run the test with proper error handling to ensure it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13228        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13750        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 14826        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 13334        |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 10414        |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15393        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15893        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14544        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17720        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 11555        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140658 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in `research/audit/5848c55fb4b2.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5848c55fb4b2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/5848c55fb4b2.tar.gz` (if generated)